SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 Data Warehousing and Data Mining

Assignment 3 – CO3

NAME: **RAGAVI S**
REG. NO : **192565027**

Course Outcome: CO3 – Understand the functionality of the various data mining components and expose them to real-world data sets to analyse their behaviour.

CO3 Coverage: *CLASSIFICATION AND PREDICTION: Issues regarding classification and prediction, decision tree induction, Bayesian classification, rule-based classification, backpropagation, support vector machines, associative classification, accuracy and error measures, evaluating a classifier or predictor (cross-validation and bootstrap), metrics for performance evaluation, prediction algorithms, and the Weka tool for mining.*

Mode: Pair Work – **2 Students per Group** | **Duration:** 1 Week | **Total Marks:** 50

Pair-Work Principle

Both students are responsible for understanding the **complete assignment**. However, each question contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must not divide the assignment question-wise. Both students must participate in every question.

The individual responsibilities intentionally alternate across questions so that both students demonstrate evidence of:

Calculate → Implement → Analyse → Evaluate

Q1 – Decision Tree Induction: Choosing the Splitting Attribute — 6 Marks

A bank records the following loan decisions:

#	Income	Credit History	Employment	Approved
1	High	Good	Salaried	Yes
2	High	Poor	Salaried	No
3	High	Good	Self-employed	Yes
4	Medium	Good	Salaried	Yes
5	Medium	Poor	Self-employed	No
6	Medium	Good	Self-employed	No
7	Medium	Poor	Salaried	No
8	Low	Good	Salaried	Yes
9	Low	Poor	Self-employed	No
10	Low	Poor	Salaried	No
11	Low	Good	Self-employed	No
12	High	Poor	Self-employed	Yes

Shared Group Work

Together:

- Identify the class attribute, the candidate splitting attributes and the stopping conditions
- Agree on the splitting criterion to be used for the manual tree and on the pruning approach

- Install Weka and load the same data (ARFF/CSV) for the experiments

Student A – Manual Induction

Student A must individually:

- Compute the **entropy** of the full data set and the **information gain** of every attribute at the root
- Compute the **gain ratio** or the **Gini index** for any one attribute and state whether it changes the choice of the root
- Build the complete tree and extract it as a set of paths
- Explain what happens if an attribute has many distinct values (for example a customer ID)

Student B – Weka Validation

Student B must individually:

- Run **J48** on the 12 tuples, and then on the *German Credit (credit-g)* data set that ships with Weka
- Compare Student A's manual tree with the Weka tree and explain every difference
- Vary the pruning confidence factor and minimum instances per leaf; report tree size and accuracy
- Explain **overfitting** using your own evidence

Individual Verification

Each student must be able to explain the other student's part of Q1.

Evidence Expected

Manual gain computations, the tree, the Weka output with parameter settings, and an interpreted comparison.

Q2 – Bayesian Classification and the Cost of Assumptions — 6 Marks

Shared Group Work

Using the loan data of Q1, classify the unseen applicant **X = (Income = Low, Credit History = Good, Employment = Self-employed)** using the Naïve Bayes classifier.

Student A – Calculation

- Compute the prior probabilities and all required conditional probabilities
- Find the posterior (proportional) value for each class and the predicted class
- Show what happens if one conditional probability is **zero**, and fix it with **Laplacian correction**; recompute

Student B – Investigation

- Run **NaiveBayes** in Weka on the same data and verify Student A's probabilities
- Create a data set (real or synthetic) in which two attributes are **strongly correlated**, and measure how the accuracy of Naïve Bayes changes compared with a decision tree
- Explain why Naïve Bayes can still give useful class rankings although the independence assumption is violated

Joint Conclusion

> **When would you prefer Naïve Bayes over a decision tree, and when would you not?**

Evidence Expected

Probability tables, posterior calculations, Laplace correction, Weka verification and evidence for the independence-assumption study.

Q3 – Rule-Based and Associative Classification — 7 Marks

Shared Group Work

Together, convert the Q1 decision tree into **IF–THEN rules** and discuss how rules represent knowledge differently from a tree.

Student A – Rule Quality

- Extract the rules and compute **coverage** and **accuracy** of each rule on the 12 tuples
- Explain rule **conflict resolution** (size ordering, rule ordering) with one example from the data
- Explain the sequential-covering idea for learning rules one at a time, and what is removed after each rule is learned

Student B – Associative Classification

- Run **JRip** (or **PART**) in Weka on *credit-g* and compare the rules with those obtained from J48
- Explain how **associative classification** (class-association rules) differs from the rules above, and what the roles of support and confidence are
- Mine class-association rules from the 12-tuple data with minimum support 25% and confidence 70%, and list those predicting *Approved* = *Yes*

Joint Design Question

> **A loan officer must be able to explain every rejection to the customer. Which of the three representations (tree, rules, associative rules) would you recommend, and why?**

Evidence Expected

Rule list with coverage/accuracy, JRip/PART output, class-association rules and a justified recommendation.

Q4 – Backpropagation: What Does the Network Learn? 7 Marks

Shared Group Work

Consider a network with 2 inputs, 2 hidden units and 1 output unit using the sigmoid activation. Inputs x1 = 1, x2 = 0, target t = 1, learning rate 0.9. Initial weights: w(x1→h1) = 0.2, w(x2→h1) = 0.4, w(x1→h2) = −0.3, w(x2→h2) = 0.1, w(h1→o) = −0.3, w(h2→o) = −0.2; biases: h1 = −0.4, h2 = 0.2, o = 0.1.

Student A – One Training Step

- Compute the forward pass: output of each hidden unit and the output unit
- Compute the error terms of the output and hidden units
- Update all weights and biases for one iteration and recompute the output; did the error decrease?
- Explain why the weights are initialised to small random values and what happens if all are set to the same value

Student B – Behaviour of the Learner

- Train a **MultilayerPerceptron** in Weka (or equivalent in Python) on a real data set
- Vary the learning rate, the number of hidden units and the number of epochs; record training and test accuracy
- Identify and demonstrate **overfitting** and **slow/unstable training**
- Compare training time and interpretability with the decision tree of Q1

Individual Verification

Each student must be able to explain how the output error propagates back to a hidden-layer weight.

Evidence Expected

Hand calculations for one iteration, tables of experiments and a discussion of strengths and limitations of neural classifiers.

Q5 – Support Vector Machines: Margin, Kernels and Scaling 7 Marks

Shared Group Work

Six points in 2-D: Class +1 → (3,3), (4,3), (4,4); Class −1 → (1,1), (1,2), (2,1). Together, plot them and discuss what a **maximum-margin** separator means.

Student A – Geometry

- Identify by inspection the **support vectors**, and write a separating hyperplane $w \cdot x + b = 0$ that is equidistant from the two classes
- Compute the margin width for your hyperplane
- Explain what changes if a non-support-vector point is removed, and what changes if a support vector is removed
- Explain the role of the **soft margin parameter C** and of **kernels** for non-linearly separable data

Student B – Experiments

- Using **SMO** in Weka (or SVC in Python), classify a non-linear data set (for example concentric circles or a real data set) with linear, polynomial and RBF kernels
- Study the effect of C and kernel parameters on accuracy and the number of support vectors
- Show the effect of **not scaling the attributes** and then of scaling them

Joint Conclusion

> **For which type of data would you choose an SVM, and what practical limitations did your experiments reveal?**

Evidence Expected

Plot with support vectors, hyperplane and margin, kernel/parameter experiments and the effect of scaling.

Q6 – Evaluating Classifiers and Predictors — 7 Marks

Shared Group Work

A fraud-detection classifier is tested on 1,000 transactions and produces: **TP = 40, FN = 20, FP = 30, TN = 910**. Together, discuss which measure is meaningful for this problem.

Student A – Metrics and Prediction Error

- Compute accuracy, error rate, **sensitivity (recall), specificity, precision and F-measure**
- Show that a classifier that predicts “not fraud” for every transaction obtains a high accuracy, and explain the **class-imbalance problem**
- For actual values {12, 15, 20, 25} and predicted values {10, 18, 19, 30}, compute **MAE, MSE, RMSE** of a predictor and explain when RMSE is preferred over MAE
- Explain how a **ROC curve** is constructed and how to read it

Student B – Estimation Methods

- On an imbalanced data set (for example *credit-g* or a fraud data set), compare **holdout, 10-fold cross-validation and bootstrap (.632)** estimates of accuracy
- Repeat holdout with at least five random splits and report the variation in accuracy
- Compare J48, Naïve Bayes and SMO using paired evaluation (for example Weka Experimenter) and report whether the differences are meaningful
- Plot a ROC curve for one classifier

Individual Verification

Each student must be able to explain why accuracy alone is misleading and which estimation method they would trust and why.

Evidence Expected

Metric calculations, error measures, estimation-method comparison with variation and an interpreted classifier comparison.

DAY-4 DESIGN CHALLENGE

Q7 – Design a Classification Solution from First Principles — 10 Marks

Problem: A hospital wants to identify patients likely to be readmitted within 30 days. Only 6% of patients are readmitted. Missing a high-risk patient (false negative) is judged **ten times more costly**

than flagging a patient who is not at risk (false positive). Doctors must be able to understand the reason for each alert. Data: age, diagnosis group, length of stay, number of prior admissions, lab results and discharge medications (some values missing).

The Day-4 problem must be solved **collaboratively**. Neither student may work on this question independently and simply combine the answers later.

Stage 1 – Independent Design Proposal

Before discussing the solution with each other, **Student A** and **Student B** each independently propose:

- The classification technique(s) they would use
- Preprocessing and handling of imbalance and missing values
- The evaluation measure and estimation method
- How the model's decisions will be explained to doctors

Each student's initial proposal must be retained in the submission. **No marks are deducted merely because an initial design is incorrect.** The purpose is to provide evidence of **independent thinking**.

Stage 2 – Compare the Two Designs

Discuss where the designs agree and differ, the strengths and weaknesses of each, missing cases, and which ideas should be retained. Record this comparison.

Stage 3 – Joint Final Design

Develop the final solution, identifying the chosen technique, preprocessing, parameters, evaluation method and cost-sensitive decision rule.

Stage 4 – Divide the Validation Responsibilities

Student A – Analytical Validation

- Compute the expected cost of two alternative decision thresholds using a confusion matrix and the 10:1 cost ratio
- Explain why accuracy is an inappropriate selection measure here
- Explain the strengths and limitations of the chosen technique for this data

Student B – Experimental Validation

- Implement the final design and a simple baseline in Weka/Python on a public analogue data set (for example a diabetes-readmission or imbalanced medical data set)
- Evaluate with cross-validation using recall, precision and ROC/AUC, and total cost
- Show the effect of the imbalance-handling method (for example resampling or cost-sensitive learning)

Stage 5 – Cross-Defence

Student A must explain:

> **What experimental evidence supports the group's design?**

Student B must explain:

> **How were the cost ratio and the decision threshold derived from the confusion matrix?**

Stage 6 – Joint Evaluation

Together determine why the problem is difficult for standard accuracy-driven classifiers, what you sacrificed to obtain interpretable alerts, and under what conditions the design would fail. Conclude:

> **What characteristics of a data set and a business requirement should a data miner examine before choosing a classifier?**

Evidence Expected

Both independent proposals, a design-comparison record, the final design, analytical and experimental validation, and a joint evaluation.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Decision Tree Induction	**6**
Q2	Bayesian Classification	**6**
Q3	Rule-Based and Associative Classification	**7**
Q4	Backpropagation	**7**
Q5	Support Vector Machines	**7**
Q6	Evaluating Classifiers and Predictors	**7**
Q7	**Day-4 Design Challenge – Classification Solution**	**10**
TOTAL		**50**

CONTRIBUTION RECORD

Each pair must include the following table at the end of the submission.

Question	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Q1			
Q2			
Q3			
Q4			
Q5			
Q6			
Q7			

The contribution record must reflect the actual work completed. **Primary contribution does not imply exclusive responsibility. Both students remain accountable for the complete assignment.**

INDIVIDUAL VERIFICATION

Faculty may select **any question**, including a component for which the student was not designated as the primary contributor. Each student must be able to:

- Explain the calculations and the tool settings used
- Interpret the evaluation measures and experimental results
- Justify design decisions
- Respond to a modified data set or requirement

A student should not answer:

> **"My partner did this section."**

NOTE: RECOMMENDED FACULTY VERIFICATION MODEL (VIVA)

For each student, conduct a brief **4–5 minute verification**. Ask:

Question 1 – Primary responsibility: one technical question from the component for which the student had primary responsibility.

Question 2 – Partner's responsibility: one question from a component primarily completed by the partner.

Question 3 – Transfer question: modify a condition and ask the student to reason about the consequence. For example:

> **"If the readmission rate falls from 6% to 0.5%, what happens to your precision, and would you change the decision threshold?"**

“If Income were replaced by a customer ID attribute with a unique value for each tuple, what would the decision tree do and why?”

“If the SVM attributes are not scaled and one attribute is measured in thousands, what happens to the margin?”

1.1 Q1 – Decision Tree Induction (6 marks)

Shared group work

- **Class attribute:** Approved (Yes / No). 5 tuples are Yes, 7 are No.
- **Candidate splitting attributes:** Income {High, Medium, Low}, Credit History {Good, Poor}, Employment {Salaried, Self-employed}. The tuple number # is an identifier and is not a candidate.
- **Stopping conditions:** (i) all tuples at a node belong to one class; (ii) no attributes remain, so use the majority class; (iii) a branch has no tuples, so use the majority class of the parent; for Weka also a minimum number of instances per leaf (M).
- **Splitting criterion:** information gain (ID3), cross-checked with gain ratio (C4.5) and the Gini index. **Pruning:** post-pruning by pessimistic error estimation (C4.5/J48, confidence factor C = 0.25) together with a minimum leaf size M; the manual tree is built unpruned first and then discussed.
- **Weka:** the 12 tuples were saved as loan.arff (nominal attributes) and loaded in the Explorer; credit-g.arff (1000 instances, 20 attributes, 700 good / 300 bad) is the German Credit data set shipped with Weka.

Student A – Manual induction

Step 1 – Entropy of the data set. Info(D) = $-(5/12)\log_2(5/12) - (7/12)\log_2(7/12) = 0.5263 + 0.4536 =$ **0.9799 bits**.

Step 2 – Information gain at the root. Info_A(D) = Σ (|Dj|/|D|)·Info(Dj); Gain(A) = Info(D) − Info_A(D).

Attribute	Partition (Yes, No) and entropy	Info_A(D)	Gain	SplitInfo	Gain ratio	Gini after split
Income	High (3,1) 0.8113; Medium (1,3) 0.8113; Low (1,3) 0.8113	0.8113	**0.1686**	1.5850	0.1064	0.3750
Credit History	Good (4,2) 0.9183; Poor (1,5) 0.6500	0.7842	**0.1957**	1.0000	**0.1957**	**0.3611**
Employment	Salaried (3,3) 1.0000; Self-employed (2,4) 0.9183	0.9591	**0.0207**	1.0000	0.0207	0.4722

Result: Credit History has the highest gain (0.1957), so it is the root. Employment is almost uninformative at the root (0.0207).

Step 3 – Gain ratio and Gini (check). Gain ratio = Gain / SplitInfo. For Income: SplitInfo = $-3\times(4/12)\log_2(4/12) = 1.585$, so GainRatio = 0.1686/1.585 = 0.1064. For Credit History the ratio is 0.1957/1.0 = 0.1957, and for Employment 0.0207. Gini(D) = $1 - (5/12)^2 - (7/12)^2 = 0.4861$; Gini after a Credit History split = (6/12)(0.4444) + (6/12)(0.2778) = 0.3611, a reduction of 0.1250, against 0.1111 for Income and 0.0139 for Employment. **The choice of root does not change:** all three criteria select Credit History.

Step 4 – Recursive construction.

- **Credit = Good** (tuples 1,3,4,6,8,11; 4 Yes, 2 No; entropy 0.9183). Gain(Employment) = 0.9183 − 0.5×0.9183 = **0.4591**; Gain(Income) = 0.2516. Split on Employment. Salaried (tuples 1,4,8) is pure Yes. Self-employed (tuples 3,6,11: 1 Yes, 2 No) is split on Income: High → Yes (tuple 3), Medium → No (tuple 6), Low → No (tuple 11).
- **Credit = Poor** (tuples 2,5,7,9,10,12; 1 Yes, 5 No; entropy 0.6500). Gain(Income) = 0.6500 − (2/6)(1.0) = **0.3167**; Gain(Employment) = 0.1909. Split on Income. Medium (tuples 5,7) and Low (tuples 9,10) are pure No. High (tuples 2,12: one No, one Yes) is split on Employment: Salaried → No, Self-employed → Yes.

```
Credit History
|-- Good
|   |-- Employment = Salaried ..... Yes   (3/3)
|   `-- Employment = Self-employed
|       |-- Income = High ............... Yes   (1/1)
|       |-- Income = Medium ............. No    (1/1)
|       `-- Income = Low ................ No    (1/1)
`-- Poor
    |-- Income = Medium ................. No    (2/2)
    |-- Income = Low ..................... No    (2/2)
    `-- Income = High
        |-- Employment = Salaried ........ No    (1/1)
        `-- Employment = Self-employed ... Yes   (1/1)
```

Step 5 – Tree as a set of paths (IF–THEN rules). Training accuracy is 12/12 = 100%.

Rule	Condition	Class	Tuples covered
R1	Credit = Good AND Employment = Salaried	Yes	1, 4, 8
R2	Credit = Good AND Employment = Self-employed AND Income = High	Yes	3
R3	Credit = Good AND Employment = Self-employed AND Income = Medium	No	6
R4	Credit = Good AND Employment = Self-employed AND Income = Low	No	11
R5	Credit = Poor AND Income = Medium	No	5, 7
R6	Credit = Poor AND Income = Low	No	9, 10
R7	Credit = Poor AND Income = High AND Employment = Salaried	No	2
R8	Credit = Poor AND Income = High AND Employment = Self-employed	Yes	12

Step 6 – Attribute with many distinct values (customer ID). An ID attribute with 12 unique values splits the data into 12 pure partitions, so Info_ID(D) = 0 and Gain(ID) = 0.9799, the maximum possible, beating every real attribute. The tree would test the ID at the root and have one leaf per tuple: 100% training accuracy and no generalisation, because a new customer has an unseen ID. Information gain is biased towards attributes with many values. Gain ratio penalises this through SplitInfo = $\log_2 12 = 3.585$, which gives 0.9799/3.585 = 0.273; note that in this small data set even this is still larger than the gain ratio of Credit History (0.1957), so gain ratio only reduces the bias and does not remove it. Practical remedies: drop identifier attributes, use binary splits (CART/Gini with subsets), enforce a minimum number of instances per leaf, prune, and validate with cross-validation.

Student B – Weka validation

J48 on the 12 tuples. Settings and results (Explorer → Classify → Use training set):

Settings	Tree	Leaves / size	Training accuracy
J48 −C 0.25 −M 2 (default, pruned)	CreditHistory = Good: [Employment = Salaried → Yes (3); Self-employed → No (3/1)]; CreditHistory = Poor → No (6/1)	3 / 5	83.33% (10/12)
J48 −U −M 1 (unpruned, no minimum)	Identical to Student A's manual tree (same attributes in the same order; 8 leaves)	8 / 13	100%

Comparison with the manual tree and explanation of every difference.

1. **Root and second level agree.** Weka also picks Credit History at the root and Employment under Good, and Income under Poor. The gain-ratio values for the three root candidates (0.1957, 0.1064, 0.0207), for Employment versus Income under Good (0.459 against 0.159) and Income versus Employment under Poor (0.200 against 0.191) give the same ordering as plain information gain.
2. **Unpruned J48 reproduces the manual tree exactly**, which validates the hand calculation.
3. **Default J48 has a smaller tree.** (a) Under Good and Self-employed, splitting on Income would create three leaves with one tuple each; M = 2 forbids a split that does not give at least two branches with two instances, so the node stays a leaf labelled No (3/1). (b) Under Poor, the Income = High split would create two one-tuple leaves; pessimistic error pruning (C = 0.25) also judges the whole Poor subtree no better than one leaf, so it collapses to No (6/1). The price is two training errors (tuples 3 and 12), giving 83.3% instead of 100%. The pruned tree is more general; the manual tree memorises the 12 tuples.
4. **Different stopping rule.** The manual tree stops only at purity; J48 also uses the minimum-leaf constraint and the error-based pruning.

J48 on credit-g (1000 instances), 10-fold cross-validation (seed 1). Tree size = number of nodes. Train = accuracy on the training set (resubstitution).

Setting	Leaves	Tree size	Train acc (%)	10-fold CV acc (%)
−C 0.05 −M 2	12	18	76.0	70.0
−C 0.10 −M 2	16	25	77.6	70.5
−C 0.25 −M 2 (default)	103	140	85.5	70.5
−C 0.25 −M 5	62	84	82.6	70.9
−C 0.25 −M 20	21	28	76.7	72.8
−C 0.50 −M 1	242	342	93.2	69.1
−C 0.50 −M 20	33	45	78.1	73.0
−U −M 2 (unpruned)	–	466	94.0	68.3
−U −M 1 (unpruned, fully grown)	–	725	100.0	66.4

Reading the table. A larger confidence factor (less pruning) and a smaller M make the tree larger and raise the training accuracy, from 76% to 100%, while the cross-validated accuracy does not improve and falls to 66.4%. Raising M from 2 to 20 at C = 0.25 shrinks the tree from 140 to 28 nodes and improves CV accuracy from 70.5% to 72.8%. Differences of less than about 1 point are within the seed-to-seed variation (standard deviation about 0.6 points over five seeds for the default J48), so state the trend and not the third digit.

Overfitting from our own evidence. The fully grown tree (725 nodes) is 100% right on the data it was built from but only 66.4% right on unseen folds: a gap of 33.6 points. The pruned small tree (18 nodes) is 76.0% on training and about 70% in CV, a gap of about 6 points. The tree that fits the training data best generalises worst, because its extra branches model noise and chance patterns, in the same way as the customer-ID tree and the 8-leaf loan tree that memorises 12 tuples.

Individual verification

Each student should be able to redo the other's part: A can read the J48 output and explain why M = 2 prunes the Self-employed branch; B can recompute the gain of Credit History (0.1957) by hand.

1.2 Q2 – Bayesian Classification and the Cost of Assumptions (6 marks)

Student A – Calculation

Unseen applicant X = (Income = Low, Credit History = Good, Employment = Self-employed). Naive Bayes chooses the class that maximises $P(Ci)\cdot\prod P(xk \mid Ci)$, the denominator P(X) being common to both classes.

Priors: P(Yes) = 5/12 = 0.4167; P(No) = 7/12 = 0.5833.

Conditional probability	Given Yes (5 tuples)	Given No (7 tuples)
P(Income = Low | C)	1/5 = 0.2000	3/7 = 0.4286
P(Credit = Good | C)	4/5 = 0.8000	2/7 = 0.2857
P(Employment = Self-employed | C)	2/5 = 0.4000	4/7 = 0.5714
P(X | C) = product	0.2 × 0.8 × 0.4 = 0.0640	0.4286 × 0.2857 × 0.5714 = 0.0700
P(X | C)·P(C)	0.0640 × 0.4167 = **0.02667**	0.0700 × 0.5833 = **0.04082**

Prediction: 0.04082 > 0.02667, so X is classified **Approved = No**. Normalised, P(No | X) = 0.04082/(0.02667 + 0.04082) = 0.605 and P(Yes | X) = 0.395.

Zero conditional probability and Laplacian correction. In the 12 tuples every attribute value occurs with each class, so X itself has no zero. To show the effect, suppose tuple 8 (the only Low-income Yes tuple) were missing: the Yes class then has 4 tuples and P(Income = Low | Yes) = 0/4 = 0. The Yes score becomes (4/11) × 0 × (3/4) × (2/4) = **0** whatever the other attributes say, so one unseen combination wipes out all the other evidence. Laplacian correction adds 1 to every count (and the number of values of the attribute to the denominator):

Term (tuple 8 removed)	Yes class (4 tuples)	No class (7 tuples)
P(Low | C)	(0+1)/(4+3) = 0.1429	(3+1)/(7+3) = 0.4000
P(Good | C)	(3+1)/(4+2) = 0.6667	(2+1)/(7+2) = 0.3333
P(Self | C)	(2+1)/(4+2) = 0.5000	(4+1)/(7+2) = 0.5556
× prior (4/11, 7/11)	0.3636 × 0.0476 = **0.01732**	0.6364 × 0.0741 = **0.04714**

The zero is replaced by a small positive probability; the Yes score is now non-zero and the decision (No) is made on all three attributes. Applying the same correction to the full 12-tuple data (also smoothing the priors to 6/14 and 8/14, which is what Weka does) gives Yes = 0.4286 × (2/8)(5/7)(3/7) = 0.03279 and No = 0.5714 × (4/10)(3/9)(5/9) = 0.04233, or P(Yes | X) = 0.437 and P(No | X) = 0.563. The class is still No.

Student B – Investigation

Weka verification. NaiveBayes on loan.arff (training set) shows the Laplace-smoothed counts: Income Yes {High 4, Medium 2, Low 2}, No {2, 4, 4}; Credit History Yes {Good 5, Poor 2}, No {3, 6}; Employment Yes {Salaried 4, Self-employed 3}, No {4, 5}; priors 0.43 / 0.57. These counts are Student A's counts plus 1, so the probabilities agree (e.g. P(Low | Yes) = 2/8). The distribution Weka returns for X is **Yes 0.437, No 0.563**, matching the Laplace calculation above and the predicted class No.

Independence-assumption study. A synthetic two-class data set of 5000 instances was generated with binary attributes: a0 agrees with the class 65% of the time, b agrees 80% of the time (independent of a0), and k further attributes are near-copies of a0 (98% identical, correlation 0.96). Weka NaiveBayes and J48 were compared with 10-fold CV repeated five times.

Copies of a0 (k)	Naive Bayes accuracy (%)	Naive Bayes AUC	J48 accuracy (%)	J48 AUC
0 (independent)	79.58	0.839	79.58	0.784
1	79.58	0.839	79.58	0.784
2	65.94	0.787	79.58	0.784
4	65.48	0.783	79.58	0.784
8	65.50	0.784	79.58	0.784

Interpretation. Naive Bayes multiplies the evidence of every attribute as if it were independent. When a weaker attribute (a0) is repeated, its evidence is counted several times and outvotes the stronger independent attribute b, so accuracy falls by about 14 points (79.6% to 65.5%). The tree tests a0 or b once and is unaffected by the duplicates, so it stays at 79.6%.

Why Naive Bayes still ranks well. The AUC of Naive Bayes stays high (0.78–0.84) although accuracy falls. The probabilities it outputs are distorted (too extreme because the evidence is double counted), but they still increase when the true risk increases, and classification and ranking only need the order of the scores or which class has the larger posterior, not the exact values. Dependencies also often affect both classes in the same direction and cancel out. This is why Naive Bayes is competitive on text and many other tasks even when independence is clearly violated; its weakness is calibration, not ranking.

Joint conclusion

- **Prefer Naive Bayes** when data are few or high dimensional (text, sparse features), attributes are roughly conditionally independent or redundancy is mild, training must be very fast and incremental, probabilistic ranking is needed, or there are many missing values (they are simply skipped).
- **Prefer a decision tree (or avoid Naive Bayes)** when attributes are strongly correlated or interact (for example XOR-type effects), when the exact calibrated probability matters, when the rules must be shown to people, or when there is enough data to learn interactions.

1.3 Q3 – Rule-Based and Associative Classification (7 marks)

Shared group work – trees versus rules

The rules R1–R8 of Q1 are mutually exclusive and exhaustive because they come from tree paths. A tree is a single hierarchical structure in which each test is conditioned on all earlier tests, so the same attribute may be tested repeatedly in different subtrees. A rule set is a flat, modular list: each rule can be read, checked, edited and added independently, and rules learned directly (sequential covering) may overlap, which leads to conflicts that need a resolution scheme. Rules can also be simplified by dropping redundant conditions.

Student A – Rule quality

Coverage = (tuples satisfying the condition)/12; accuracy = (covered tuples with the predicted class)/(covered tuples).

Rule	Covered	Correct	Coverage	Accuracy
R1 Good ∧ Salaried → Yes	3	3	25.0%	100%

Rule	Covered	Correct	Coverage	Accuracy
R2 Good ∧ Self ∧ High → Yes	1	1	8.3%	100%
R3 Good ∧ Self ∧ Medium → No	1	1	8.3%	100%
R4 Good ∧ Self ∧ Low → No	1	1	8.3%	100%
R5 Poor ∧ Medium → No	2	2	16.7%	100%
R6 Poor ∧ Low → No	2	2	16.7%	100%
R7 Poor ∧ High ∧ Salaried → No	1	1	8.3%	100%
R8 Poor ∧ High ∧ Self → Yes	1	1	8.3%	100%

The coverages add up to 100% and every rule is 100% accurate on the training data, but five rules cover one tuple each, so their accuracy is not trustworthy. The tree rules never conflict; conflicts arise once rules are learned or simplified independently.

Rule conflict resolution – example from the data. Take two independent rules: **RA:** Credit = Good ∧ Employment = Self-employed → No (covers tuples 3, 6, 11; 2 of 3 correct, accuracy 67%) and **RB:** Income = High → Yes (covers tuples 1, 2, 3, 12; 3 of 4 correct, accuracy 75%). Tuple 3 (High, Good, Self-employed) fires both rules with different classes.

- **Size ordering:** the rule with the most attribute tests (the toughest requirement) fires. RA has two tests, RB has one, so RA fires and predicts No, which is wrong because tuple 3 is Yes.
- **Rule ordering:** rules are sorted in a decision list by a priority (accuracy, coverage or class), and the first matching rule fires. If the list is ordered by accuracy, RB (75%) comes before RA (67%), so the prediction is Yes, which is correct here. If no rule fires, a default rule (majority class No) is used.

Sequential covering. Rules are learned one class at a time and one rule at a time (AQ, CN2, FOIL, RIPPER). Procedure: (1) start with an empty rule set; (2) learn the best rule for the target class by greedily adding the attribute test that most improves quality (accuracy or FOIL gain) until the rule is accurate enough; (3) **remove the tuples covered by the new rule**, both the correctly classified positives and the covered negatives, from the training set; (4) repeat with the remaining tuples until no positives are left or the rule quality falls below a threshold; (5) add a default rule. Removing the covered tuples makes the next rule specialise on what is still unexplained. Example: the rule Credit = Good ∧ Salaried → Yes covers tuples 1, 4, 8; they are removed, and the next rule is learned from the remaining 9 tuples.

Student B – Associative classification

JRip and PART on credit-g (10-fold CV, seed 1) compared with J48:

Algorithm	CV accuracy (%)	Model size	Example of what it says
J48 (−C 0.25 −M 2)	70.5	103 leaves, 140 nodes	Deep tree starting with checking_status = <0 then foreign_worker, duration, job, purpose…
JRip (−F 3 −N 2 −O 2)	71.7	**3 rules**	(checking_status = <0) AND (job = skilled) → bad (172/76); (checking_status = 0<=X<200) AND (duration >= 24) AND (savings_status = <100) → bad (61/19); otherwise good (767/162)
PART (−M 2 −C 0.25)	70.2	78 rules in a decision list	checking_status = no checking AND other_payment_plans = none AND credit_history = critical → good (134/3); …

Comparison. The three models have nearly the same accuracy, but the sizes differ greatly. JRip finds a very compact description (it grows rules with FOIL gain and prunes them with reduced-error pruning and a MDL

stopping rule) and pays with a default rule that accounts for most data; PART builds a partial tree for each rule, so its rules resemble tree paths and are ordered; J48 gives the most complete but the largest model. Checking account status appears in all three, which shows it is the strongest signal. On the 12 loan tuples JRip gives Income = High → Yes; Credit = Good ∧ Salaried → Yes; otherwise No.

How associative classification differs. Rule learners such as RIPPER, CN2 and tree paths search greedily and keep a small set of rules that together cover the data. Associative classification (CBA, CMAR, CPAR) first mines **all** class-association rules (itemset → class) that pass the thresholds, using an Apriori/FP-growth search over frequent itemsets, then ranks them and selects a small subset to build the classifier. It therefore considers combinations of attributes that a greedy search would never test, and it can reveal several alternative explanations. **Support** = percentage of all tuples that contain the antecedent and the class (how often the pattern occurs; it removes rare, chance patterns). **Confidence** = support of antecedent-and-class divided by support of the antecedent (the rule's accuracy on the tuples it covers). Rules are ranked by confidence, then support, then shorter length; prediction uses the first matching rule or a weighted vote of the matching ones.

Class-association rules at min support 25% (at least 3 tuples out of 12) and min confidence 70%. Mining was done by hand and checked with Weka Apriori (−A class association rules, −M 0.25, −C 0.7); both give the same six rules:

#	Rule	Support	Confidence	Predicts
1	Credit = Good ∧ Employment = Salaried → Approved = Yes	3/12 = 25%	3/3 = 100%	**Yes**
2	Income = High → Approved = Yes	3/12 = 25%	3/4 = 75%	**Yes**
3	Credit = Poor ∧ Employment = Salaried → No	3/12 = 25%	3/3 = 100%	No
4	Credit = Poor → No	5/12 = 41.7%	5/6 = 83.3%	No
5	Income = Medium → No	3/12 = 25%	3/4 = 75%	No
6	Income = Low → No	3/12 = 25%	3/4 = 75%	No

Rules predicting Approved = Yes: Credit = Good ∧ Employment = Salaried → Yes (25%, 100%) and Income = High → Yes (25%, 75%). Note that Credit = Good alone (support 4/12, confidence 4/6 = 67%) fails the 70% threshold.

Joint design question – explaining rejections

Recommendation: ordered or independent IF–THEN rules, preferably high-confidence associative rules shown as a single "why" rule. A rejected customer needs one short, understandable reason. In a tree the reason is the path from the root, which can be long and may include tests irrelevant to the customer (the full tree on credit-g has 103 leaves). Rules are self-contained, so the officer can quote the firing rule, e.g. "Credit History = Poor, which led to rejection in 5 of 6 past cases", and can state its support and confidence, which gives a measure of reliability. Associative rules are the best for explanation because they are exhaustive over strong patterns and have measured support and confidence, but they can be numerous and overlapping, so they must be pruned, ranked and have a conflict policy (highest confidence first). A tree is acceptable when it is small. Whatever is chosen should be checked for legally sensitive attributes and for stability.

2.4 Q4 – Backpropagation (7 marks)

Student A – One training step

Network 2-2-1, sigmoid O = 1/(1 + e^−I), x1 = 1, x2 = 0, t = 1, learning rate l = 0.9. Weights: w(x1→h1) = 0.2, w(x2→h1) = 0.4, w(x1→h2) = −0.3, w(x2→h2) = 0.1, w(h1→o) = −0.3, w(h2→o) = −0.2; biases θh1 = −0.4, θh2 = 0.2, θo = 0.1.

Forward pass.

Unit	Net input I	Output O
h1	0.2(1) + 0.4(0) − 0.4 = −0.2000	1/(1+e^0.2) = **0.4502**
h2	−0.3(1) + 0.1(0) + 0.2 = −0.1000	1/(1+e^0.1) = **0.4750**
o	−0.3(0.4502) − 0.2(0.4750) + 0.1 = −0.1301	1/(1+e^0.1301) = **0.4675**

Error terms (Err_j = O_j(1 − O_j)(T_j − O_j) for the output; Err_j = O_j(1 − O_j)Σ Err_k w_jk for hidden units):

- Err_o = 0.4675 × 0.5325 × (1 − 0.4675) = **0.1326**
- Err_h1 = 0.4502 × 0.5498 × (0.1326 × −0.3) = **−0.00984**
- Err_h2 = 0.4750 × 0.5250 × (0.1326 × −0.2) = **−0.00661**

Updates (Δw = l × Err_j × O_i; Δθ = l × Err_j):

Parameter	Calculation	Old	New
w(h1→o)	−0.3 + 0.9(0.1326)(0.4502)	−0.3000	**−0.2463**
w(h2→o)	−0.2 + 0.9(0.1326)(0.4750)	−0.2000	**−0.1433**
θo	0.1 + 0.9(0.1326)	0.1000	**0.2193**
w(x1→h1)	0.2 + 0.9(−0.00984)(1)	0.2000	**0.1911**
w(x2→h1)	0.4 + 0.9(−0.00984)(0) – unchanged because x2 = 0	0.4000	**0.4000**
θh1	−0.4 + 0.9(−0.00984)	−0.4000	**−0.4089**
w(x1→h2)	−0.3 + 0.9(−0.00661)(1)	−0.3000	**−0.3060**
w(x2→h2)	unchanged because x2 = 0	0.1000	**0.1000**
θh2	0.2 + 0.9(−0.00661)	0.2000	**0.1940**

Recompute the output. h1: net = 0.1911 − 0.4089 = −0.2177, O = 0.4458. h2: net = −0.3060 + 0.1940 = −0.1119, O = 0.4721. o: net = −0.2463(0.4458) − 0.1433(0.4721) + 0.2193 = 0.0418, O = **0.5105**. The output moved from 0.4675 to 0.5105 (towards the target 1); the error T − O fell from 0.5325 to 0.4895 and the squared error ½(T − O)² from 0.1418 to 0.1198. **Yes, the error decreased** (by about 15%).

Why small random initial weights. (i) Random values break the symmetry: if all weights are equal, every hidden unit computes the same output and receives the same error and the same update, so they stay identical forever and the hidden layer behaves like one unit however many are used. (ii) Small values keep the net inputs near zero, where the sigmoid is steep and gradients are large; large weights saturate the sigmoid (O close to 0 or 1), O(1 − O) becomes almost 0 and learning stalls (vanishing gradient). (iii) Different random starts help to reach different local minima.

Student B – Behaviour of the learner

Weka MultilayerPerceptron on the diabetes data set (768 instances, 8 numeric attributes, 2 classes) with a 66%/34% random split (seed 1); attributes are normalised by Weka. Momentum 0.2. J48 on the same split: train 78.5%, test 76.2%, 0.07 s.

Learning rate (H = 4, 500 epochs)	Train (%)	Test (%)	Time (s)
0.01	76.1	80.8	0.35
0.1	81.3	77.8	0.17
0.3	80.9	77.8	0.15
0.9	80.7	79.7	0.19

Hidden units (L = 0.3, 500 epochs)	Train (%)	Test (%)	Time (s)
1	76.3	78.5	0.06
2	79.5	78.9	0.09
4	80.9	77.8	0.13
10	86.2	73.9	0.35
30	87.0	73.9	0.65
60	88.6	73.9	1.53

Epochs (L = 0.3, 30 hidden units)	Train (%)	Test (%)	Time (s)
10	75.3	80.5	0.02
50	77.1	80.8	0.08
100	80.1	78.9	0.17
500	87.0	73.9	0.79
2000	95.9	73.2	3.25
5000	95.9	73.6	8.05

Overfitting. With 30 hidden units the training accuracy climbs from 75.3% (10 epochs) to 95.9% (2000 epochs) while the test accuracy falls from about 80.8% to 73.2%: the network is memorising noise. More hidden units show the same pattern (training 88.6%, test 73.9% with 60 units). **Slow training.** A learning rate of 0.01 needs many more epochs to reach what 0.1 reaches in few (training accuracy 76.1% against 81.3% after 500 epochs), and training time grows linearly with epochs and roughly with the number of weights (8.05 s for 5000 epochs). Very large rates and momentum make the error oscillate and may diverge; on this small data set the rate 0.9 still worked, so test unstable settings (rate > 1 or momentum near 1) on your own run to demonstrate it. Single-split test accuracies on 261 instances have a standard error of about 2.5 points, so use repeated splits before trusting small differences.

Comparison with the decision tree. The network needs normalisation, parameter tuning (rate, momentum, hidden units, epochs) and is several times to more than a hundred times slower to train (0.15–8 s against 0.07 s for J48), and its accuracy (about 78–80% test) is not clearly better than the tree's 76% on this data. The tree is interpretable (rules), whereas the network is a black box with weights that cannot be read as reasons. Neural networks are strong when many numeric inputs interact non-linearly and large amounts of data exist; they are limited by local minima, overfitting, long training and low interpretability.

Individual verification – error propagation

The output error (T − O) is multiplied by the output derivative O(1 − O) to give Err_o. That error is sent backwards through each connection in proportion to the weight (Err_o × w(h→o)), multiplied by the hidden unit's own derivative h(1 − h), to give Err_h. The weight update for an input-to-hidden connection is then l × Err_h × x_i. Thus a hidden weight changes more if the output error is large, the connection to the output is strong, the hidden unit is not saturated and the input is non-zero (which is why w(x2→h) did not change).

2.5 Q5 – Support Vector Machines (7 marks)

Shared group work

A maximum-margin separator is the hyperplane that separates the classes and is as far as possible from the nearest training points of each class; maximising the margin 2/||w|| gives the lowest generalisation bound, and the solution depends only on the points on the margin (the support vectors).

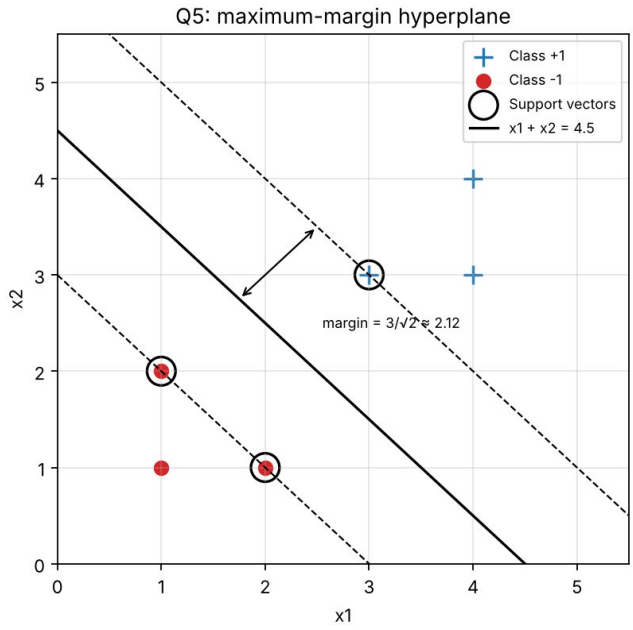


Figure 1. Six points, support vectors (circled), hyperplane x1 + x2 = 4.5 and margin boundaries.

Student A – Geometry

Support vectors by inspection. The closest opposite-class pairs are (3,3) with (2,1) and (3,3) with (1,2) (distance √5 = 2.236 each, the shortest). So the support vectors are **(3,3) for class +1 and (2,1) and (1,2) for class −1**. The points (4,3), (4,4) and (1,1) are further back.

Hyperplane. By the symmetry in x1 and x2, w = (a, a). On the margins w·x + b = +1 at (3,3) and −1 at (2,1): 6a + b = 1 and 3a + b = −1, so a = 2/3, b = −3. Check on (1,2): 2/3 + 4/3 − 3 = −1 ✓. Hence w = (2/3, 2/3), b = −3, i.e. **(2/3)x1 + (2/3)x2 − 3 = 0, equivalent to x1 + x2 = 4.5**. The other points satisfy the constraints: (4,3) gives +1.67, (4,4) gives +2.33, (1,1) gives −1.67. The Lagrange multipliers are α = 4/9 for (3,3) and 2/9 for each of (2,1) and (1,2): they are all positive and Σαy = 0, which confirms optimality. The line is equidistant from the two classes: the distance from each support vector is 1/||w|| = 1.061. A solver gives the same result (scikit-learn SVC with a very large C: w = (0.667, 0.667), b = −3.00, 3 support vectors).

Margin width = 2/||w|| = 2/(√(4/9 + 4/9)) = 2/(2√2/3) = 3/√2 = **2.12** units.

Effect of removing points. Removing a non-support vector such as (4,4) changes nothing: w, b and the margin stay the same (verified with the solver: identical hyperplane). Removing a support vector, for example (3,3), changes the solution: the next closest positive point (4,3) becomes a support vector, the

hyperplane moves to x1 + x2 = 5 (w = (0.5, 0.5), b = −2.5), and the margin widens to 2.83. A hard-margin SVM is therefore decided by very few points, which is also its sensitivity to outliers and noise.

Soft margin C and kernels. For data that are not perfectly separable, slack variables $\xi_i \geq 0$ allow violations and the objective becomes minimise $\frac{1}{2}||w||^2 + C\Sigma\xi_i$. A **large C** penalises errors heavily: a narrow margin, few training errors, risk of overfitting. A **small C** accepts more errors: a wider margin, smoother boundary, risk of underfitting. For non-linearly separable data a kernel $K(x, z) = \phi(x)\cdot\phi(z)$ computes the dot product in a higher-dimensional feature space without constructing it (the kernel trick), so a linear separator there is a curved boundary in the original space. Common kernels: polynomial $(x\cdot z + 1)^d$ and RBF $\exp(-\gamma||x - z||^2)$; γ controls how local the influence of each point is.

Student B – Experiments

Data: 600 points on two concentric circles with noise 0.12 (scikit-learn make_circles), 33% test split. (The same study can be repeated with SMO in Weka using PolyKernel and RBFKernel.)

Kernel (C = 1)	Train acc	Test acc	Support vectors
Linear	0.629	0.571	400
Polynomial d = 2	0.993	0.990	34
Polynomial d = 3	0.995	0.990	24
RBF γ = 1	0.993	0.985	60

The linear kernel is no better than chance because circles are not linearly separable, and all 400 training points become support vectors; the polynomial and RBF kernels reach 98.5–99% test accuracy with far fewer support vectors.

C (RBF, γ = 1)	Train	Test	Support vectors
0.01	0.993	0.985	402
0.1	0.993	0.985	190
1	0.993	0.985	60
10	0.998	0.990	22
100	1.000	0.990	13
1000	1.000	0.985	9

γ (RBF, C = 1)	Train	Test	Support vectors
0.01	0.714	0.662	402
0.1	0.980	0.990	311
1	0.993	0.985	60
10	0.995	0.985	71
100	1.000	0.995	282
1000	1.000	0.874	390

Effect of C and γ. As C grows, the margin narrows, the training accuracy rises to 100% and the number of support vectors falls from 402 to 9 (a hard margin keeps only the points that define the boundary). A very small γ (0.01) underfits (66% test), a very large γ (1000) makes each point its own island: training accuracy 100% but test accuracy drops to 87.4% with 390 support vectors, a typical overfitting sign. Choose C and γ together by cross-validated grid search.

Effect of not scaling. The second attribute of the same circles data was multiplied by 1000 (as if one attribute were measured in thousands):

Model (C = 1)	Unscaled: train / test / SV	Standardised: train / test / SV
RBF γ = 1	0.998 / 0.641 / 398	0.998 / 0.985 / 51
RBF γ = 'scale'	0.796 / 0.843 / 203	0.995 / 0.985 / 51
Linear	0.639 / 0.591 / 400	0.629 / 0.571 / 400

Without scaling, the large-range attribute dominates the distance $||x - z||^2$ (and the dot products), so the other attribute is practically ignored; the RBF model memorises the training set (99.8%) but achieves only 64.1% on the test set. After standardising, the test accuracy is 98.5%. In Weka, SMO normalises by default (−N 0); setting it to −N 2 switches that off. On the margin: a unit of the large attribute counts a thousand times more than a unit of the small one, so the margin is determined almost entirely by the large attribute.

Joint conclusion

Choose an SVM for medium-sized data with many numeric features where classes are separable with a clear margin, including high-dimensional sparse data such as text and gene expression, and when a non-linear boundary is needed (RBF or polynomial kernels). Practical limitations seen in the experiments: the results depend strongly on scaling, C and γ (tuning by grid search is needed); the kernel matrix makes training slow on very large data ($O(n^2)$–$O(n^3)$); the model is not interpretable; there are no calibrated probabilities by default; and nominal attributes and class imbalance need extra handling.

2.6 Q6 – Evaluating Classifiers and Predictors (7 marks)

Shared group work

With 1000 transactions of which 60 (6%) are fraud, the meaningful measures are recall (sensitivity: how many frauds are caught), precision (how many alerts are real frauds) and their combination (F-measure, ROC/AUC, or total cost), not plain accuracy.

Student A – Metrics and prediction error

Confusion matrix: TP = 40, FN = 20, FP = 30, TN = 910. P = TP + FN = 60, N = FP + TN = 940, total 1000.

Measure	Formula	Value
Accuracy	(TP + TN)/(P + N) = 950/1000	**95.0%**
Error rate	(FP + FN)/(P + N) = 50/1000	**5.0%**
Sensitivity (recall)	TP/P = 40/60	**66.7%**
Specificity	TN/N = 910/940	**96.8%**
Precision	TP/(TP + FP) = 40/70	**57.1%**
F-measure (F1)	2PR/(P + R) = 2(40)/(2(40) + 30 + 20) = 80/130	**0.615**

All "not fraud" classifier. TP = 0, FN = 60, FP = 0, TN = 940, so accuracy = 940/1000 = **94%**, only one point below the real classifier (95%), although its recall is 0% and it catches no fraud at all. This is the **class-imbalance problem**: the majority class dominates accuracy, so a high value says little about the rare class that matters. Remedies: use recall, precision, F-measure, AUC or cost-based measures; stratified sampling; resampling (over- or under-sampling, SMOTE); cost-sensitive learning; moving the decision threshold.

Prediction error. Actual {12, 15, 20, 25}, predicted {10, 18, 19, 30}; errors (actual − predicted) = 2, −3, 1, −5.

Measure	Calculation	Value
MAE	(2 + 3 + 1 + 5)/4 = 11/4	**2.75**
MSE	(4 + 9 + 1 + 25)/4 = 39/4	**9.75**
RMSE	√9.75	**3.12**

MAE or RMSE. RMSE squares the errors before averaging, so large errors weigh much more (the error of 5 contributes 25 of the 39). RMSE is preferred when large errors are disproportionately costly (and should be avoided), when errors are roughly normally distributed, and when a differentiable loss is needed for optimisation. MAE is preferred when the error cost is linear and the data have outliers that should not dominate. RMSE ≥ MAE always; the larger the gap (3.12 against 2.75), the more uneven the errors.

ROC curve. For a classifier that outputs a score or probability, sort the test tuples by decreasing score of the positive class. Start at (0, 0) and go through the list: for an actual positive move up by 1/P (the true-positive rate rises), for an actual negative move right by 1/N (the false-positive rate rises). Each threshold is one point (FPR, TPR). **Reading:** the diagonal is random guessing (AUC = 0.5); the closer the curve is to the top-left corner (0, 1), the better; AUC is the probability that a random positive is ranked above a random negative; points on one curve show the trade-off between catching more positives and raising more false alarms, so the threshold can be chosen for the costs involved. Our classifier is a single point at FPR = 30/940 = 3.2%, TPR = 66.7%.

Student B – Estimation methods

Data: credit-g, 1000 instances (700 good, 300 bad), J48 default (−C 0.25 −M 2). The class split of 70/30 is moderately imbalanced; for strong imbalance repeat the study on a fraud data set.

Holdout (66/34) split seed	1	2	3	4	5	Mean ± SD
Accuracy (%)	72.65	73.53	74.41	68.82	71.76	72.24 ± 2.15 (range 68.8–74.4)

Estimation method (J48)	Estimate (%)	Variation
Holdout, 5 random splits	72.24	SD 2.15 across splits, range 5.6 points
10-fold cross-validation, 5 different seeds	71.30 (70.5, 72.3, 71.2, 71.2, 71.3)	SD 0.64
Bootstrap .632 (50 samples)	75.52 = 0.368 × 85.50 (resubstitution) + 0.632 × 69.72 (out-of-bag)	Out-of-bag SD 2.15

Interpretation. Holdout varies strongly with the split (a 5.6-point range) because it uses only 340 test cases and a smaller training set; 10-fold CV is much more stable (SD 0.64) and uses all data for both training and testing. The .632 bootstrap gives a higher value (75.5%) here because the resubstitution accuracy of J48 (85.5%) is very optimistic and is weighted by 0.368; it is useful for small data sets but is known to be biased for classifiers that overfit strongly. **Method trusted:** stratified 10-fold cross-validation, repeated with different seeds (10 × 10-fold), because of its low variance and low bias; for tiny data sets, bootstrap; holdout only when data are very large.

Paired comparison of J48, Naive Bayes and SMO (10 repetitions of stratified 10-fold CV on the same folds, i.e. 100 paired results per algorithm, with the corrected resampled t-test used by the Weka Experimenter, significance 0.05):

Algorithm	Mean accuracy (%)	SD	Compared with J48
J48	71.25	3.17	–
Naive Bayes	75.16	3.48	+3.91, t = 2.78, p = 0.0065 → significantly better
SMO (linear)	75.10	3.46	+3.85, t = 2.95, p = 0.0039 → significantly better

Naive Bayes against SMO: difference 0.06 points, p = 0.95, so **not meaningful**. Conclusion: NB and SMO are both about 4 points more accurate than J48 on credit-g and this is statistically significant, but NB and SMO cannot be separated. In the Experimenter, set the test base to J48 and "Percent_correct", 10 repetitions, 10-fold CV and look for the "*" or "v" marks.

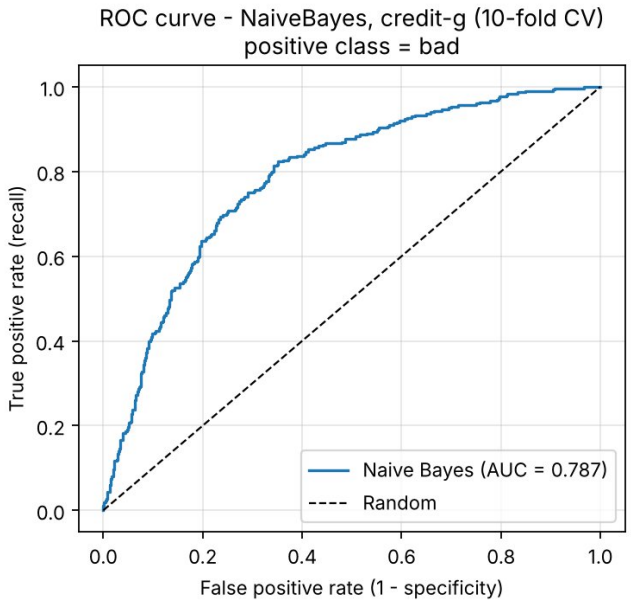


Figure 2. ROC curve of Naive Bayes on credit-g (positive class = bad), AUC = 0.787.

Reading the curve: at threshold 0.5 the point is TPR 0.50, FPR 0.14; lowering the threshold to 0.3 gives TPR 0.70, FPR 0.25; at 0.1 TPR 0.88, FPR 0.49. The curve lies clearly above the diagonal (AUC 0.787), so the ranking is useful, and the threshold can be set according to the cost of missing a bad customer.

Individual verification

Why accuracy alone is misleading: it counts all errors equally and is dominated by the majority class (94% for a classifier that detects nothing), and it hides the type of error. **Which estimation method to trust:** stratified, repeated 10-fold cross-validation, because its estimates have low variance and low bias, and because it allows paired significance tests.

2.7 Q7 – Day-4 Design Challenge: Hospital Readmission (10 marks)

Problem: 6% of patients are readmitted within 30 days; a false negative costs 10 times a false positive; doctors need a reason for every alert; the data (age, diagnosis group, length of stay, prior admissions, labs, discharge medications) contain missing values.

Stage 1 – Independent design proposals (retain in the submission)

> The two proposals below show the expected level of detail and are deliberately different. Replace them with the real proposals that each of you wrote independently before discussing; no marks are lost if they differ from this key or are partly wrong.

Element	Student A's initial proposal	Student B's initial proposal
Technique	Logistic regression with regularisation, because it gives a probability and coefficients that doctors can read.	Pruned decision tree or rule learner (J48/PART) with a limited depth, because the path is the explanation.
Preprocessing, imbalance, missing values	Median imputation plus a "missing" indicator for each lab test; one-hot encoding of diagnosis group; class weights to counter the 6% rate.	Trees handle missing values by fractional instances; oversample the readmitted class (SMOTE) in the training folds only.
Evaluation measure and estimation	Recall at a fixed precision, ROC-AUC and expected cost; stratified 10-fold CV.	Recall and F-measure; stratified 10-fold CV.
Explanation to doctors	Top three factors by coefficient × value for each patient; odds ratios in a table.	The rule path that fired, with the number of patients and the readmission rate in that leaf.

Stage 2 – Comparison of the two designs

Aspect	Agreement / difference	Strengths and weaknesses; what to keep
Common ground	Both reject plain accuracy, use stratified cross-validation, want an explanation per alert, treat imbalance explicitly.	Keep all four.
Model	A: logistic regression; B: tree/rules.	LR: smooth, calibrated probabilities, handles many weak signals, easy threshold; but cannot model interactions unless added. Tree: captures interactions and gives a path; but unstable, high variance on 6% positives, coarse probabilities (leaf frequencies) that make a threshold such as 1/11 hard to apply. **Keep LR as the scoring model.**
Imbalance	A: class weights; B: SMOTE.	Resampling can distort probabilities and, done before the split, leaks information; a cost matrix is directly tied to the 10:1 requirement. **Keep cost-sensitive learning/thresholding; test resampling as an alternative.**
Missing values	A: imputation plus indicator; B: native handling in the tree.	Missingness itself may be informative (a test not ordered). **Keep imputation plus indicators.**
Missing from both	Calibration of probabilities, a monitoring plan, fairness check, repeated admissions of the same patient across folds (group-wise CV).	Add to the final design.

Stage 3 – Joint final design

- **Technique:** regularised logistic regression (ridge) as the scoring model; a shallow decision tree trained on the model's alerts is used only to communicate typical high-risk profiles.
- **Preprocessing:** median imputation (fitted on training folds only) plus missing-value indicators; standardise numeric inputs; one-hot encode diagnosis group and discharge medications; no resampling before the split.
- **Imbalance and cost:** cost matrix FN = 10, FP = 1 (TP = TN = 0). Because the model outputs a probability p, **flag the patient if $p \geq C_{FP}/(C_{FP} + C_{FN}) = 1/11 \approx 0.091$** (not 0.5).
- **Evaluation:** repeated stratified 10-fold CV grouped by patient; report recall, precision, ROC-AUC (and PR-AUC) and **total cost = 10 × FN + 1 × FP**; select the model and threshold by cost, not accuracy.
- **Explanation:** each alert lists the three largest contributions (coefficient × standardised value) with plain-language labels, plus the odds-ratio table for the model.

Derivation of the threshold. For a patient with readmission probability p, flagging costs (1 − p) × 1 in expectation (a false alarm if not readmitted) and not flagging costs p × 10. Flag when (1 − p) < 10p, i.e. p > 1/11 = 0.0909. At a 6% base rate this threshold is above the average risk, so only patients with at least about 1.5 times the average risk are flagged.

Stage 4 – Validation

Student A – Analytical validation

Expected cost of two thresholds (illustration on 1000 patients, 60 readmitted). Cost = 10 × FN + 1 × FP. The confusion matrices are illustrative values for a typical model; replace them with those from your own experiment.

Threshold	TP	FN	FP	TN	Recall	Precision	Total cost
0.50	24	36	20	920	40%	54.5%	10(36) + 20 = **380**
0.09 (= 1/11)	48	12	188	752	80%	20.3%	10(12) + 188 = **308**
Reference: flag nobody	0	60	0	940	0%	–	600
Reference: flag everybody	60	0	940	0	100%	6%	940

The default threshold of 0.5 has the higher accuracy (94.4% against 80.0%), yet it costs 380 against 308, an expected saving of 72 cost units per 1000 patients (19%) when the threshold is lowered. The lower threshold accepts many more false alarms (188 against 20) because each one costs only one tenth of a miss.

Why accuracy is inappropriate. It treats FN and FP as equally costly and is dominated by the 94% non-readmitted: a model that flags nobody has 94% accuracy and the worst clinical value (cost 600); in the table the higher-accuracy threshold is the more expensive one. Selection should use expected cost, recall at a clinically acceptable precision, and AUC.

Strengths and limitations of the chosen technique. Strengths: stable with few positives (60 per 1000), gives a probability that can be thresholded for any cost ratio, readable coefficients, fast, and easy to monitor. Limitations: assumes a linear effect on the log-odds (interactions and thresholds in labs must be added by hand), sensitive to correlated predictors and to imputation, needs calibration if resampling or weighting is used, and the explanation shows association, not causation.

Student B – Experimental validation

Public analogue data: the Weka diabetes data (Pima), with the 268 positives subsampled to 32 and 500 negatives retained (532 patients, 6.0% positive); physiologically impossible zeros in glucose, blood pressure, skin thickness, insulin and BMI were set to missing (438 cells). Weka, 10 × 10-fold stratified CV, average confusion matrix (32 positives, 500 negatives). Cost = 10 × FN + FP.

Model	TP	FN	FP	Recall	Precision	AUC	Cost
ZeroR (flag nobody)	0.0	32.0	0.0	0.00	–	0.48	320.0
Baseline: J48 default	0.3	31.7	5.4	0.01	0.05	0.52	322.4
Logistic, default threshold 0.5	0.7	31.3	5.5	0.02	0.11	0.79	318.5
J48 + Resample (50:50 oversampling)	8.4	23.6	57.3	0.26	0.13	0.59	293.3
Logistic + Resample (50:50)	20.8	11.2	119.6	0.65	0.15	0.78	231.6
J48 + CostSensitive (10:1 reweighting)	9.5	22.5	54.1	0.30	0.15	0.62	279.1
Logistic + CostSensitive (10:1 reweighting)	17.4	14.6	83.1	0.54	0.17	0.78	229.1
Final design: Logistic + CostSensitive (min. expected cost = threshold 1/11)	16.9	15.1	76.1	0.53	0.18	–*	**227.1**

** The AUC of a CostSensitiveClassifier that outputs hard decisions is not meaningful; the underlying logistic model has AUC 0.79.*

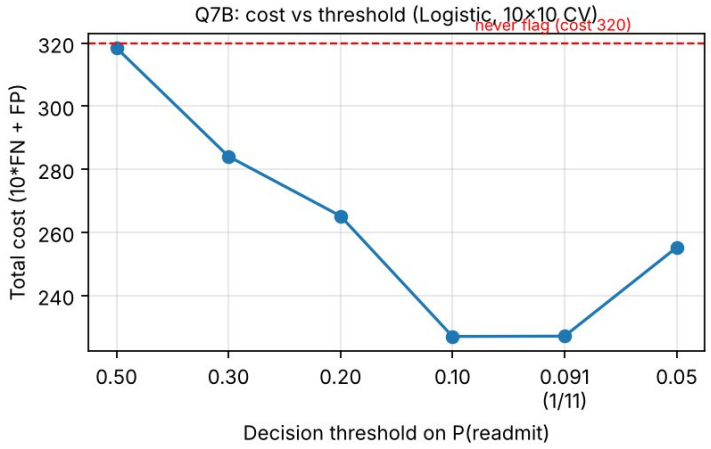


Figure 3. Total cost against decision threshold for logistic regression (10 × 10-fold CV).

Findings. (1) The default J48 and the default 0.5 threshold essentially flag nobody: recall 1–2%, cost 318–322, the same as ZeroR (320), although accuracy is about 93%. (2) The logistic model ranks well (AUC 0.79) but its default threshold is wrong for this cost ratio; lowering the threshold to 1/11 gives recall 0.53, precision 0.18 and cost 227, **about 30% below the baseline (322)**. (3) Resampling and cost-sensitive reweighting give similar costs (232 and 229) for logistic regression; the tree profits less (279–293) and its AUC remains low (0.59–0.62), which supports the choice of the logistic model. (4) The cost curve in Figure 3 is flat near 0.09–0.10 and rises on both sides, consistent with the theoretical 1/11 optimum. Caution: the data set has only 32 positives, so results vary with the sample (a few positives change the cost by tens of units); this is an analogue, not hospital data.

Stage 5 – Cross-defence

Student A explains the experimental evidence: the cost fell from 322 (default J48) or 320 (flag nobody) to 227 for the final design, recall rose from 1% to 53%, precision is only about 18%, so roughly 8 of 10 alerts are false alarms, which is acceptable only because a miss costs ten times more, and logistic regression beat the tree in AUC (0.79 against 0.52–0.62).

Student B explains the cost ratio and threshold: from the confusion matrix each patient falls into one of four cells with costs TP = 0, TN = 0, FP = 1, FN = 10. The expected cost of flagging is $(1 - p) \times 1$ and of not flagging $p \times 10$; they are equal at $p = 1/11 = 0.091$, which is the threshold. In the Stage-4 illustration, moving the threshold from 0.50 to 0.09 reduces FN from 36 to 12, saving $24 \times 10 = 240$, and raises FP from 20 to 188, adding 168; the net saving is 72 (380 → 308).

Stage 6 – Joint evaluation

- **Why it is hard for accuracy-driven classifiers:** with 6% positives the accuracy-optimal rule is to predict the majority class; a 10:1 cost asymmetry reverses the optimum; missing data and weak signals make the ranking imperfect (AUC about 0.8 even in our analogue), so any threshold needs many false alarms to catch half of the true cases.
- **What we sacrificed for interpretable alerts:** a linear model cannot capture interactions between labs, medications and prior admissions, so some predictive power (AUC) may be given up compared with boosted trees or neural networks; precision is low (about 18%), so alert fatigue is a risk; coarse, simple explanations may omit true drivers.
- **Conditions under which the design fails:** a large shift in prevalence (for example from 6% to 0.5%: recall unchanged, but precision falls roughly twelve-fold, so the threshold must be re-tuned); a change

in the cost ratio; changes in coding or lab practice (data drift) and in the missingness pattern; poorly calibrated probabilities; small samples for rare diagnoses; and leakage of post-discharge information.

What should a data miner examine before choosing a classifier? Data characteristics: size, number and type of attributes (numeric, nominal, text), missing values and noise, class balance, correlation and interaction among attributes, dimensionality, and whether the data are time-ordered or grouped by patient. Business requirements: unequal costs of the two errors, need for explanation, required probability or ranking output, legal and fairness constraints, speed of training and scoring, and how the model will be monitored and updated. These determine the evaluation measure and the model family, not the other way round.